Decorator

Season: 2 · Structural · Category: GoF · Level: Beginner · Build with Waqar

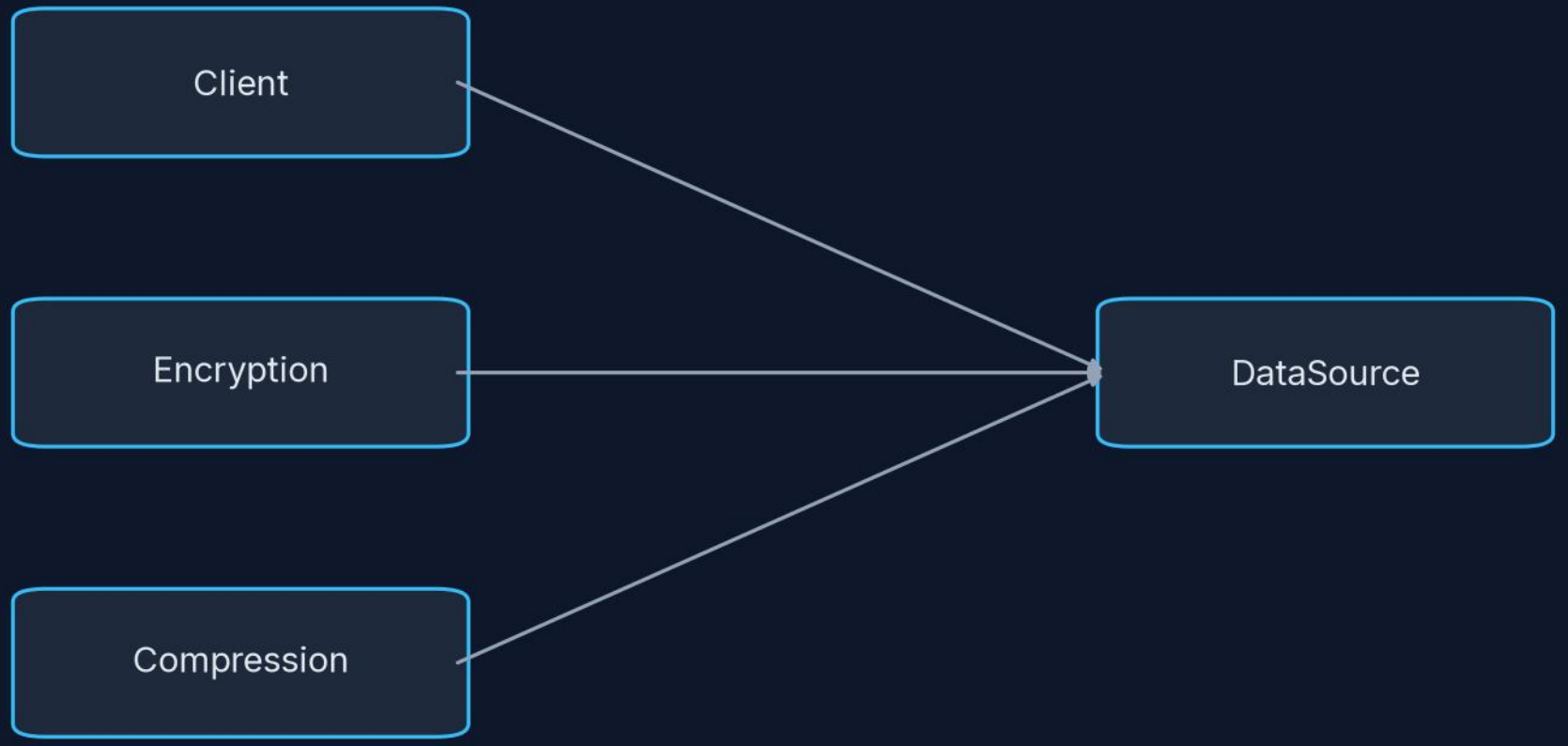
Today's Hook

- Today: Decorator as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Decorator structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Decorator
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/gof/structural/decorator/DataSourceDecorators.java
package com.buildwithwaqar.patterns.gof.structural.decorator;

interface DataSource {
    String read();
    void write(String data);
}

final class InMemoryDataSource implements DataSource {
    private String data = "";
    public String read() { return data; }
    public void write(String data) { this.data = data; }
}

abstract class DataSourceDecorator implements DataSource {
    protected final DataSource wrappee;
    protected DataSourceDecorator(DataSource wrappee) { this.wrappee = wrappee; }
    public String read() { return wrappee.read(); }
    public void write(String data) { wrappee.write(data); }
}

final class EncryptionDecorator extends DataSourceDecorator {
    EncryptionDecorator(DataSource wrappee) { super(wrappee); }
```

Implementation (2)

```
    @Override public void write(String data) {
        super.write(new StringBuilder(data).reverse().toString());
    }

    @Override public String read() {
        return new StringBuilder(super.read()).reverse().toString();
    }
}

/** Stackable decorator demo entry — compression over encryption over memory. */
public final class DataSourceDecorators {
    private DataSourceDecorators() {}

    public static DataSource encryptedCompressedMemory() {
        return new EncryptionDecorator(new CompressionDecorator(new InMemoryDataSource()));
    }
}

final class CompressionDecorator extends DataSourceDecorator {
    CompressionDecorator(DataSource wrappee) { super(wrappee); }

    @Override public void write(String data) {
        super.write("c:" + data.replace(" ", ""));
    }
```

Implementation (3)

```
    @Override public String read() {
        var s = super.read();
        return s.startsWith("c:") ? s.substring(2) : s;
    }
}
```

Demo.main

```java
package com.buildwithwaqar.patterns.gof.structural.decorator;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Decorator walkthrough ===");
    DataSource ds = DataSourceDecorators.encryptedCompressedMemory();
    ds.write("hello world");
    System.out.println("readBack=" + ds.read());
    DataSource raw = new InMemoryDataSource();
    raw.write("plain");
    System.out.println("raw=" + raw.read());
  }
}
```

Walkthrough Output

```
=== Decorator walkthrough ===
readBack=helloworld
raw=plain
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Decorator
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Decorator → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.